校园创意验证 Agent

面向校园服务创意的多智能体验证与编排系统

参赛方向： Agent 编排创新应用

参赛形式： 个人参赛

版本： 1.0

日期： 2026 年 10 月 31 日

摘要

“校园创意验证 Agent”是一套面向校园服务早期创意的本地可运行多智能体应用。它帮助使用者把一句模糊的问题描述，转化为一份包含目标用户、待验证假设、访谈问题、最小验证方案、可观察指标和风险提醒的行动草案。系统重点解决创意讨论中常见的证据不足、计划不可执行、角色分工不清和生成过程难以复核等问题。

系统采用轻量化的 Python 与 Streamlit 架构。规划 Agent 与方案 Agent 接收同一份不可变输入快照，在标准库线程池中并行执行；两项结果完成后，独立验证 Agent 检查方案草稿的结构和证据表达；真实模型模式下，验证未通过时 Self-Loop 将问题清单交给修订步骤，最多返工两次。每个并行角色最多重试两次，所有单次异常和最终耗尽状态都会记录。失败后返回明确的安全空值，不把失败结果伪装成通过。结构化运行轨迹记录任务标识、角色状态、重试次数、耗时和失败原因，便于检查一次运行发生了什么。

应用提供模拟模式和真实模型 API 模式。模拟模式使用本地确定性逻辑，适合无密钥演示和流程检查，页面及导出结果都会标明模拟身份。真实模式通过兼容 Chat Completions 的接口调用用户配置的模型服务。项目当前没有真实 API 实测数据，也没有用户试点、部署规模或成效数据，因此不作相应效果承诺。内置知识指南采用本地关键词检索，不宣称为向量数据库或外部事实核查。

本项目的工程重点是一个小而完整、能够观察和复现的多智能体链路：任务输入、能力匹配、并行执行、有界重试、独立验证、有限反馈、结果导出和运行追踪。现有实现并非通用 Agent 平台，不包括可视化工作流编辑器、在线动态注册、MCP 外部工具或实时资源调度。本文将已实现功能、测试观察和后续计划分开叙述，避免将设计目标写成已经取得的结果。

关键词： 多智能体编排；并行调度；Self-Loop；能力匹配；失败重试；运行轨迹；校园创新

一、项目背景与场景需求

1.1 场景

校园服务创意通常从具体经历开始，例如新生找不到校园卡补办流程、社团活动报名入口分散、课程资料通知不集中，或者学生希望更方便地预约公共空间。提出创意并不等于验证了需求。早期项目很容易跳过问题确认，直接进入功能列表和页面开发。团队随后才发现目标用户不明确、问题出现频率不清楚、已有替代方法没有调查，或者原定指标无法观察。

个人参赛者还要在有限时间内完成需求梳理、方案设计、测试准备、代码实现和展示材料。单次大模型问答虽然可以快速生成文字，却难以呈现任务如何拆分、不同角色如何分工、验证意见怎样影响下一轮输出、发生异常时系统如何处理。项目因此需要一条可读、可追踪、可重复运行的工作流。

本应用把任务范围收窄为“校园服务创意的早期验证准备”。它不代替真实访谈，不判断一个项目是否应当创业，也不自动生成可被当成事实的用户证据。它的作用是协助使用者先写清楚要验证什么、找谁验证、观察什么结果，并把生成过程拆成可检查的步骤。

1.2 目标用户与待完成任务

首要用户是有校园服务或校园内容创意的学生个人参赛者。典型任务是输入问题、目标用户和已有证据，拿到一份短周期验证计划。课程实践中的学生小组也可以使用同一流程整理访谈提纲和原型测试方案。指导教师可以检查学生是否区分了观察事实与模型建议，但当前版本没有教师账号、班级管理或批量评阅功能。

用户需要完成的实际动作包括：

- 用自己的话描述校园问题，不只填写预想功能。
- 指定希望服务的人群和目前掌握的证据。
- 查看系统提出的关键假设与调查问题。
- 检查方案是否把未经验证的内容写成事实。
- 根据验证问题修改方案，或先开展访谈后再运行一次。
- 导出结果及运行记录，供后续复盘。

1.3 需求分析与设计原则

一是帮助用户把问题写具体。输入界面要求说明需求与目标用户，并允许填写已有证据。证据为空时，系统应引导用户去收集信息，而不是补造调研结果。

二是把任务分解给不同 Agent。规划和方案起草是两个互不依赖的子任务，可以从同一份用户输入分别开始；验证任务需要看到两者的结果，因此在它们完成后执行；修订任务需要验证问题清单，因此只在验证不通过时执行。

三是使每次模型调用有清楚的边界。并行任务有固定重试上限，验证反馈环有最大迭代次数。遇到异常时，日志显示失败原因和尝试序号，最终失败用安全空值表示，调用者可以看见流程没有成功生成该部分内容。

四是使模拟与真实模型运行可以区分。模拟模式用于开发、界面讲解和流程演练。它不访问模型，不代表模型质量。真实模式使用本地环境变量中的 API 配置，演示时需要让页面明确显示当前模式。

五是把生成建议和实地证据分开。模型可以协助形成假设、问题和观察指标，但真实用户反馈、使用数据、市场规模和成效必须来自实际调研或可靠来源。

二、平台总体架构

2.1 技术组成

应用使用 Python 3 编写业务逻辑，以 Streamlit 提供本地网页界面。系统无需独立前端工程、数据库或云端服务即可启动。角色定义保存在 JSON 注册表中；本地验证指南保存在 JSON 知识文件中；一次运行的结构化轨迹保存在本地 runs 目录。模型客户端使用 Python 标准库处理兼容接口请求，因此没有新增第三方 Agent 框架依赖。

模块	职责
app.py	收集输入、展示模式、结果、日志及下载入口
core.py	构造输入快照、调用角色、验证结果、执行 Self-Loop、写入轨迹
scheduler.py	读取 Agent 注册表、按能力匹配候选角色、并行运行独立任务及执行有界重试
agent_registry.json	保存角色职责、能力标签和静态调度配置
knowledge_base.json	保存本地项目验证指南，供关键词匹配
trace_view.py	查看最近一次运行轨迹的终端工具
test_core.py	使用标准库 unittest 的自动化测试
README.md	环境准备、启动方法、运行模式和项目说明

2.2 数据流与执行图

每次运行从用户填写的问题、目标用户与证据开始。编排器将这些值整理为一个不可变的输入快照，并为规划任务与方案任务分别创建调用。两个角色不读取对方的中间输出，因此可以并行运行。

执行流程： 用户输入 → 规划与方案并行 → 验证方案 → 必要时修订（最多两轮）→ 输出结果与日志。

图中只有规划和方案两个角色并行。验证与修订存在明确的数据依赖，按顺序执行。当前工作流是在代码中定义的固定流程，不是用户可视化编辑的 DAG 平台；也没有跨运行持久化的旧结果回滚机制。

2.3 Agent 角色与输入输出

规划 Agent（planner）读取输入快照中的问题、目标用户、证据与本地指南，提出目标陈述、关键假设、访谈问题、短周期行动步骤、成功指标和风险。它应把缺失证据标注为待验证内容。

方案 Agent（solution）独立读取同一份输入快照和本地指南，整理形成验证方案草稿。输出包含问题与证据、最小可行尝试、用户接触方式、观察指标和风险说明。为了保持并行，该角色不等待规划 Agent 的结果。验证器以方案草稿、原始需求、目标用户和证据为输入检查关键字段与表达；规划结果单独展示给使用者。

独立验证 Agent（reviewer）在规划和方案输出就绪后执行。它检查必要信息是否齐全、假设是否与事实区分、验证动作是否可执行、指标是否可以观察。真实模式中它通过单独的模型调用返回结构化验证结果；模拟模式中它执行本地确定性规则。

Self-Loop 修订步骤（reviser）在真实模型模式下接收原方案和验证问题清单，针对缺项或不清楚的内容修订。修订后再次验证。它最多执行两轮；达到上限仍不通过时，系统保留失败状态和问题提示，不输出“已通过”的假结论。

2.4 状态、结果与异常处理

角色状态分为完成、失败和最终耗尽等可区分状态。并行角色每次单独尝试失败时，日志记录角色、尝试编号和异常摘要。最多两次重试意味着每个并行角色最多调用三次。任一尝试成功后返回该次结果，并保留此前的失败记录；所有尝试都失败后返回预设安全空值和最终异常。最终耗尽汇总与单次尝试错误分别记录，二者不混为一次错误。

“安全空值”只表示当前角色没有提供可用输出，不代表回滚到上一次成功运行。编排状态在一次调用内创建，没有跨运行持久化的稳定快照。这一边界使失败语义清楚，也避免界面把空结果误认作有效方案。

Self-Loop 的重试与并行角色的异常重试是两种不同机制：前者由验证未通过触发，按验证问题修改内容；后者由调用抛出异常触发，对同一角色调用进行最多两次额外尝试。网络异常重试不会被当作质量修订，质量修订也

不会改变并行阶段的尝试次数。

三、Agent 编排与调度算法

3.1 注册表和能力匹配

角色注册表为每个 Agent 保存稳定标识、显示名称、职责、能力标签、优先级、经验分、配置负载和版本等字段。调度器根据当前任务内容提出能力需求，再与注册表中的角色能力做匹配。规划任务会关注用户研究、证据和指标等能力；方案任务会关注产品设计、流程和指标；验证任务会关注证据表达和完整性。

对候选角色，调度逻辑综合能力覆盖、经验分、优先级和静态负载配置形成排序。运行记录中保留派发理由，使开发者能够检查某个任务为何选择该角色。注册表中的负载是人工配置值，不是实时 CPU 使用率、API 并发数或剩余配额。注册表可通过编辑 JSON 调整，应用没有在线注册管理页面，也不在运行期间动态发现服务。

3.2 并行执行策略

并行调度器位于 scheduler.py，使用 Python 标准库 concurrent.futures.ThreadPoolExecutor。调度器接收角色到调用函数的映射以及失败时的安全返回值映射，为每个独立角色创建 Future，再按提交次序收集结果。因此规划 Agent 与方案 Agent 可以重叠执行，但结果字典仍按输入角色顺序组织，调用方通过角色名读取输出。

线程池工作数不小于同时执行的角色数，使两个任务具备并发运行条件。当前并行范围刻意限定在没有数据依赖的规划与方案两项任务。独立验证需要合并两者结果，所以在并行阶段完成后开始。修订需要验证结果，所以只在失败分支进入。这是一个可解释的局部并行工作流，不声称支持任意规模的图调度、任务优先级抢占或分布式执行。

并行减少的是两个独立模型请求的串行等待时间。理论上，当二者都成功时，等待时间更接近较慢请求的耗时，而非两次耗时相加；真实收益取决于模型服务的排队、网络、并发限制与速率限制。目前没有真实模型耗时对照，因此本文不报告具体提速比例。

3.3 有界重试与失败处理

每个并行角色第一次异常后，调度器最多再尝试两次，总尝试数上限为三。重试次数可通过调度器参数控制；系

统默认采用两次额外重试。每次异常以类型和文本摘要记录，并附尝试序号。当某次调用成功时，返回成功值及总尝试数，同时保留此前失败详情。当所有尝试都失败时，返回显式失败标记、安全空值、总尝试数和最后异常。

失败追踪同时写入页面运行日志和 JSON 轨迹。日志中的单次失败条数等于实际失败尝试数。最终耗尽汇总单独使用耗尽状态，确保统计重试次数时不会把汇总行当作额外尝试。写入运行轨迹失败不会中断用户当前看到的主流程。

对同一角色调用进行重试不能消除模型服务持续故障的风险。安全值是有意设计的降级结果，不提供旧运行的缓存内容，也不会自动认为另一个角色的输出可以代替失败角色。

3.4 Self-Loop 反馈逻辑

验证器生成通过状态或结构化问题列表。通过时工作流进入最终展示；未通过时，编排器将问题列表与当前草稿传给修订步骤，再调用验证器复查。最多进行两轮修订。每轮结果、问题摘要和状态都进入日志。达到上限后停止，保留仍未解决的问题。

模拟模式使用本地确定性规则检查方案结构，不执行模型修订环。真实模式下，验证器由模型给出结构化判断，未通过时编排器触发修订，并最多复查两轮。模型判断可能受提示词、模型版本和输入变化影响。系统的章节完整性检查不能证明需求真实存在，也不能证明方案合规、安全或具有商业价值；人工需要对真实结论和外部事实进行复核。

3.5 运行监控与可观测性

结构化轨迹按每次执行写入 JSON 文件，记录任务标识、运行时间、执行模式、各角色开始与结束信息、耗时、状态、失败原因和重试次数。终端查看工具可以打印最近轨迹的时间线。页面同时呈现角色日志和最终验证状态，帮助使用者发现失败发生在模型调用、验证还是修订阶段。

轨迹适合本地调试和复现，但它不是分布式追踪平台。当前不采集服务端指标、API 服务商队列、token 账单、用户行为分析或跨设备数据。轨迹文件可能包含错误摘要，因此提交公开仓库时只应保留无个人信息的示例，真实运行记录不应未经审查直接公开。

四、模型、检索与数据安全

4.1 模型服务接入

真实模式支持与 Chat Completions 兼容的服务。使用者通过环境变量设置 API 地址、模型名称和密钥。密钥只在本机环境中读取，不写入源代码、截图、运行日志或下载报告。不同服务商对模型名称、鉴权、超时和请求格式可能有差异，使用前需要以服务商文档为准。真实 API 未配置时，系统会提示配置要求，或由使用者选择模拟模式。

真实模型调用是系统中的语言理解与生成能力来源。规划、方案、验证和修订任务采用不同角色指令，使输出格式和职责区分。模型输出仍可能遗漏、混淆事实或生成不准确建议，所以系统将生成内容作为草案，并让独立验证步骤检查结构和证据措辞。当前没有模型微调，也没有声称使用了特殊训练数据。

4.2 本地知识指南

本地知识文件包含项目验证方面的短指南，调度流程按关键词匹配相关条目，并把命中内容交给规划与方案角色。该实现不向外部网站发起搜索，不建立向量索引，也不表示所有校园信息已经过来源核验。

知识条目可以由参赛者按相同字段扩充，但应为可核实的指南写明来源或适用条件。若未来接入外部检索或 MCP 工具，系统还需要保存来源地址、抓取时间、引用片段、访问权限和失败情况，并由人工确认来源是否适合支撑结论。

4.3 数据与使用边界

本地 MVP 不提供用户账号、云数据库或长期项目资料存储。用户刷新页面后可能失去当前会话状态，因此需要主动下载结果。真实模式会将必要输入发送到使用者配置的模型服务；使用前应阅读服务商的数据处理条款，只输入完成任务所需的信息，避免提交姓名、学号、手机号和未公开资料。

应用不应自动发送报名材料或发布生成内容。下载报告前，使用者应检查事实、隐私和措辞，并删除个人数据。运行轨迹用于开发复盘，不是用户行为分析数据集。项目目前不作特定司法辖区下的合规认证声明。

五、核心创新与技术贡献

5.1 两条独立工作支路并行

规划和方案生成通常都需要理解原始问题、目标用户与已有证据，但互相不依赖。系统明确以同一输入快照启动这两条支路，并在后续汇合。这样可以在保持工作流可解释的同时使用并行调度。并行范围、依赖边界和阶段顺序都清楚可见，便于展示和测试。

5.2 调度过程可解释

系统将任务能力要求与角色注册表中的能力标签匹配，并记录派发理由。开发者可以检查角色为什么被选中，也能通过修改注册表观察配置对选择结果的影响。当前的负载信息是静态配置，不能据此宣称实现了实时资源调度。

5.3 单次错误与整体耗尽分离

每次异常对应一次真实尝试，重试耗尽另有汇总状态。这个区分使日志条目、总尝试数和轨迹重试数保持一致，失败时也能保留最后异常。发生失败后明确返回安全值与错误状态，不把安全值包装成有效生成结果。

5.4 反馈环有验证信号和停止条件

真实模型模式下的 Self-Loop 由独立验证结果触发，每次修订输入可追溯的问题，最多进行两轮。到达上限后保留未通过状态，不进行无边界循环。这使反馈环具备可复现的运行上限，便于测试其分支与停止行为。

5.5 模拟运行不伪装为模型效果

无 API Key 时可以用模拟模式展示用户流程，但页面和导出结果明确标记模式。模拟流程只验证应用的输入、调度显示、日志和导出，不用于评价真实模型准确率、响应速度或用户价值。体验者可以从界面标签和报告字段辨认实际执行模式。

六、典型使用流程

以“新生补办校园卡时找不到办事入口”为例。使用者填写问题、目标用户“刚入学的新生”和已有证据；如果

还没有访谈记录，就将证据留空。系统不会将假设写成已有事实。

规划 Agent 可能生成需要验证的假设、访谈问题和三天行动计划；方案 Agent 根据同一输入并行整理信息架构原型、测试任务与风险。两项输出就绪后，验证 Agent 检查关键内容是否齐全、目标指标是否可观察、方案是否将未知内容明确标为假设。若有缺项，修订步骤按验证问题补充一次，再交给验证器检查。最多两轮后，系统展示最终状态、日志、耗时摘要和可下载材料。

使用者还需要真实联系新生，观察他们是否能找到正确入口，记录完成任务所需时间及遇到的困难。应用输出的访谈问题只是建议，不是访谈证据。是否采用某个原型、是否继续开发，应依据真实测试结果决定。

七、测试与当前验证结果

7.1 自动化测试范围

项目使用 Python 标准库 unittest 编写核心测试，并通过 pytest 启动同一测试集。测试覆盖模拟流程输出、验证与 Self-Loop 次数上限、API 请求结构和缺少密钥提示、并行角色结果隔离、失败后成功重试、三次尝试全部失败时的安全值与日志、结构化轨迹状态，以及 planner 与 solution 的并发重叠。

并发测试利用 threading.Barrier 和 Event 控制两个角色的同步点。只有两个角色都进入调用后，Barrier 才能继续；测试由此检查二者确实同时执行，而不是仅依据最终结果推断并发。失败路径测试检查每次单独异常的日志数量与尝试编号，并检查最终耗尽行和轨迹重试数没有重复计数。

7.2 测试结果

当前工作区以 pytest 执行 test_core.py，13 项测试全部通过。该结果验证的是本地自动化测试中的代码路径，不代表真实 API 连通性、模型事实准确率或真实用户体验效果。测试中的模型响应使用本地替身，不会发送真实服务请求。

观察项	当前结果	结果边界
核心测试	13 项通过	本地自动化用例，不是外部验收

并发重叠	Barrier/Event 测试通过	证明测试替身执行时两个角色重叠
并行失败重试	每角色最多三次尝试	不验证服务商是否允许并发或是否限流
验证反馈上限	自动化用例覆盖	规则完整性不等于事实真实性
真实模型效果	尚未测量	不报告准确率或提速比例
真实用户试点	尚未开展	不报告满意度、节省时间或采用率

7.3 后续评测设计

真实用户测试应先取得同意，招募少量目标用户完成相同任务，记录样本数、招募方式和操作过程。调研结果只代表参与者，不外推为全校结论。方案质量可以由两位评审按假设清晰度、步骤可执行性、指标可观察性和证据区分进行评分；如只有一位评审，应如实注明。

稳定性评测应在固定模型、固定提示词版本和可控网络下重复运行，记录成功率、超时率、解析失败、重试次数、验证通过率、总耗时和模型调用次数。并行模式与串行基线应使用同一批输入、同一服务和相近运行时段，多轮报告中位数及离散程度。费用应结合服务商账单或明确的定价来源核算，不能仅用估算 token 数宣称实际成本。

验证器评测需要准备包含缺失章节、证据不足、目标用户不清、结论夸大、格式错误和通过样例的固定数据集。记录正确拦截、漏报与误报，保存人工标注规则和模型版本。在这些评测完成前，所有指标只作为计划。

八、局限、风险与应对

模型可能生成不准确内容。独立验证步骤主要检查结构和表达，不是权威事实核查。涉及校园流程、政策、价格、市场规模和安全事项的结论需要人工查证并注明来源。

并行 API 可能受到限流。服务商可能限制并发、配额或请求速率。当前角色重试设有次数上限，但没有指数退避、全局速率限制或令牌预算控制。真实运行要从小规模验证，并观察服务商返回的错误码和费用。

知识指南较简单。关键词匹配可能漏掉同义表达，也不能保证检索相关性。当前体量适合说明本地检索路径，不

适合作为覆盖全面的校园知识库。扩充资料时应整理来源、更新时间与适用范围。

编排流程固定。两个角色并行，验证和修订顺序执行。用户不能拖拽节点、定义任意依赖或动态注册远程 Agent。它是面向一个具体任务的单机原型，不是通用平台。

本地运行不等于生产部署。应用未提供账号、权限管理、数据库备份和云端监控。运行轨迹保存在本地；不应把含用户内容的文件直接公开。公开仓库只包含代码、文档和无敏感信息的示例配置。

九、开发安排与验收计划

提交前按以下顺序完成本地验收：

- 在干净环境安装 requirements.txt 中的依赖，确认应用启动。
- 先用模拟模式完成一次全流程，核对页面模式标识、结果和下载文件。
- 检查并行日志、失败尝试、最终耗尽、验证返工和轨迹文件。
- 如果配置真实模型服务，则以非敏感输入检查接口、模型名、响应结构与费用，并单独标明真实模式。
- 在另一台设备或干净虚拟环境按 README 重新运行，避免依赖开发机私有文件。
- 公开仓库前扫描密钥、个人信息、临时日志和本地缓存。
- 录制当前版本的演示视频，视频画面与代码、PPT 和报告保持一致。

计划中的用户调研、真实模型对照试验、外部工具/MCP 接入、负载测试和云部署均不作为已完成事项。开发时间优先用于真实用户任务测试、真实 API 运行记录和演示复核；能否实现由实测决定，不通过修改文案把计划写成成果。

十、项目交付内容

本项目随源码仓库提供 Python 应用、Agent 注册表、本地知识指南、核心自动化测试、运行轨迹查看工具、启动说明、方案书及演示材料。运行者可以用模拟模式复现本地流程，也可以自行配置兼容的模型 API。测试用例使用本地替身，避免在测试期间产生模型调用费用。

项目仓库地址：

https://github.com/kanghaoyue11/campus-idea-agent

提交前应由参赛者打开仓库确认最新代码、方案书、PPT、依赖清单和启动说明均已公开可读。演示视频需要使用最终代码版本录制，并单独按报名平台接受的格式提交。若仓库或平台材料与本地运行结果不同，应以实际演示版本为准及时修订。

结语

校园创意验证 Agent 将多智能体分工、并行调度、异常重试、独立验证、有限反馈和运行轨迹组合到一个可本地运行的原型中。它针对的是校园创意早期验证这一具体场景，强调让假设、证据和建议保持可区分，让失败和重试过程可以被观察。

当前实现提供了可复现的软件流程和自动化测试，但尚无真实模型性能数据和真实用户试点结论。下一步工作应从实际使用者和可复核的测试数据开始，逐步评估它是否真的帮助用户更快完成有效验证。项目价值最终取决于可运行代码、透明的实验过程和诚实的结果记录。